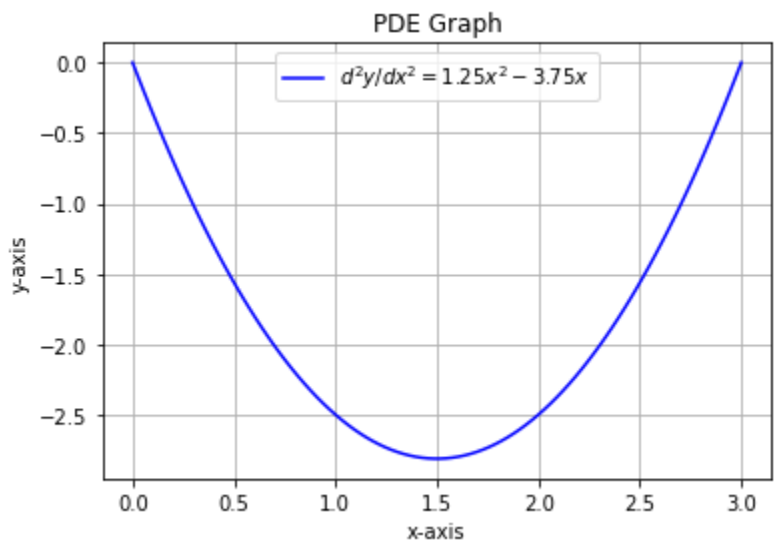
Tapia, Ramsey

Finite Beam Deflection Solver

Spring 2024

Introduction and motivation: We are given a beam with a distributed load. We are given the length of the beam, the magnitude of the distributed load, and the product of Young's modulus and cross-sectional moment of inertia. Using our matrix and vector libraries, we aim to solve for beam deflection using a 2nd-order finite-difference approximation by discretizing the beam into equispaced points.

Prep Work:



B. $EI\left(\frac{y_{i+1} - 2y_i - y_{i-1}}{\Delta x^2}\right) = \frac{-q}{2}\left(x_i^2 - x_i L\right)$

$i=0: \quad y_0 = 0$

$i=1: \quad y_2 - 2y_1 - y_0 = \frac{-q\Delta x^2}{2EI}\left(x_1^2 - x_1 L\right)$

$i=2: \quad y_3 - 2y_2 - y_1 = \frac{-q\Delta x^2}{2EI}\left(x_2^2 - x_2 L\right)$

$i=3: \quad y_4 - 2y_3 - y_2 = \frac{-q\Delta x^2}{2EI}\left(x_3^2 - x_3 L\right)$

$i=4: \quad y_4 = 0$

$$K = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 1 & -2 & 1 & 0 & 0 \\ 0 & 1 & -2 & 1 & 0 \\ 0 & 0 & 1 & -2 & 1 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{Bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ y_4 \end{Bmatrix} = \begin{Bmatrix} 0 \\ \frac{-q\Delta x^2}{2EI}\left(x_1^2 - x_1 L\right) \\ \frac{-q\Delta x^2}{2EI}\left(x_2^2 - x_2 L\right) \\ \frac{-q\Delta x^2}{2EI}\left(x_3^2 - x_3 L\right) \\ 0 \end{Bmatrix}$$

Scanned with CamScanner

Implementation: To compile the code, be sure to enter the terminal and run this on the compilation line: “g++ -o beamer beamer.cpp beam.cpp matrix.cpp vector.cpp”. Once you have compiled your code, you can run it by typing this on the compilation line: “./beamer”. When entered, you will see this output:

```
[class@235c9d13f928 proj02]$ ./beamer
---------------------------------------
Beam Solver 9003
   -> Solving a simply supported beam
---------------------------------------

Usage: beam [n] [L] [EI] [q] <turbo>

where:
  [n]            number of points
  [L]            length of the beam
  [EI]           (Young's modulus)*(2nd moment of interia)
  [q]            distributed load per unit length
  <turbo>        optional argument (1=turbo mode)
```

Using this guide, we will test the case of n = 20 - here's the output of that:

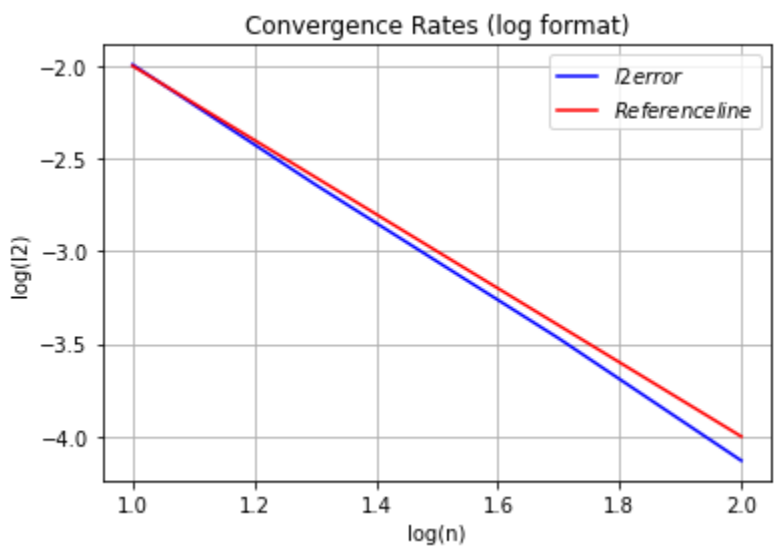
```
[class@235c9d13f928 proj02]$ ./beamer 20 3 600 1500
Solving [A]x = b using Gauss-Seidel iterative method:
  --> Iteration:     1  norm = 1
  --> Iteration:     2  norm = 0.493572
  --> Iteration:     3  norm = 0.324744
  --> Iteration:     4  norm = 0.240344
  --> Iteration:     5  norm = 0.18972
  --> Iteration:     6  norm = 0.155985
  --> Iteration:     7  norm = 0.131903
  --> Iteration:     8  norm = 0.113854
  --> Iteration:     9  norm = 0.0998263
  --> Iteration:    10  norm = 0.088614
  --> Iteration:    11  norm = 0.0794491
  --> Iteration:    12  norm = 0.0718197
  --> Iteration:    13  norm = 0.0653716
  --> Iteration:    14  norm = 0.0598515
  --> Iteration:    15  norm = 0.0550739
  --> Iteration:    16  norm = 0.0508995
  --> Iteration:    17  norm = 0.0472219
  --> Iteration:    18  norm = 0.0439583
  --> Iteration:    19  norm = 0.0410435
  --> Iteration:    20  norm = 0.038425
  --> Iteration:    21  norm = 0.0360606
  --> Iteration:    22  norm = 0.0339157
  --> Iteration:    23  norm = 0.0319617
  --> Iteration:    24  norm = 0.0301748
  --> Iteration:    25  norm = 0.0285349
Iterations: 25
Solve Time: 0.000811
l2 Error: 0.506557
Mesh Interval: 0.157895
```

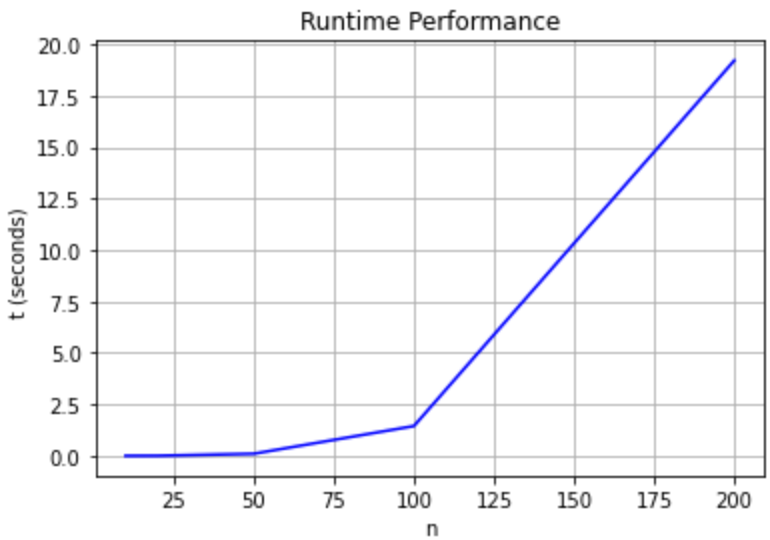
However, when I manually set the debug flag to false and increase the number of max iterations, the code does not print each iteration. It runs until it satisfies the l2 error.

Analysis Runs (non-turbo mode): include the requested table

n	*h*	*l*2 Error	# of iterations	Wall-clock time (secs)
10	0.333333	0.0101603	133	0.000309
20	0.157896	0.00227958	541	0.003466
50	0.0612245	0.000340377	3144	0.101077
100	0.030303	7.40615e-05	11439	1.44785
200	0.0150754	1.9359e-05	40620	19.2195

Verification and Runtime Performance:





Looking at the convergence plot, the line showcasing the log data for the l2 error is almost parallel with the reference line. Thus, my code is working quite fairly. To further justify my claim, looking at the graph displaying runtime performance, the time increases as expected as the number of points increases. This shows that my code is working because the l2 error decreases as the time to solve increases.